

Code-Layout, Readability and Reusability

Date	28 June 2026
Team ID	XXXXXX
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the OptiCrop codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S. No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform 4-space indentation used throughout all Python files (app.py, preprocessing.py, model.py)	Yes	PEP 8 standards followed across all Python scripts
2	Proper File Structure	Project is organized into logical folders: static/ (css, js), templates/, and root files (app.py, requirements.txt)	Yes	Clear separation of frontend, backend, and ML model files
3	Meaningful Variable Names	Variables like crop_name, feature_values, scaled_input, prediction clearly reflect their purpose	Yes	No single-letter or ambiguous variable names used
4	Function / Method Names	Functions named descriptively: validate_input(), predict_crop(), load_model(), build_features()	Yes	Snake_case naming convention followed throughout
5	Code Comments	Inline and block comments explain key logic such as feature scaling, model loading, and input validation	Yes	Docstrings added to all major functions in app.py and model scripts
6	Modular Design	Code is split into reusable modules: data preprocessing, model training, Flask routes, and input validation are all separate functions	Yes	app.py imports from separate helper modules for clean architecture
7	No Redundant Code	Duplicate or unused code removed; no dead imports or commented-out blocks left in	Yes	Code reviewed and cleaned before deployment

		final version		
8	Error Handling	Exceptions and errors handled gracefully using try/except blocks in Flask routes; descriptive error messages returned as JSON	Yes	400 and 500 HTTP error codes returned with meaningful messages

Reusable Components / Modules:

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	validate_input()	Python / Flask	Called in /predict route handler before every inference	High
2	load_model() / joblib.load()	Python / Scikit-learn	Loaded once at app startup; reused for every prediction request	High
3	StandardScaler (scaler.pkl)	Python / Scikit-learn	Applied to all incoming feature arrays before model inference	High
4	LabelEncoder (label_encoder.pkl)	Python / Scikit-learn	Used to decode numeric model output to crop name string	High
5	renderResult() — JavaScript function	JavaScript / HTML	Reused to dynamically inject crop recommendation into the DOM on every prediction	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Logical folder structure, consistent indentation, and clean file organization throughout
Readability	5	Meaningful variable and function names, inline comments, and docstrings make code easy to understand
Reusability	5	Modular design with separate validation, inference, and rendering functions that can be reused across the codebase
Documentation / Comments	4	Key functions documented with docstrings; inline comments explain complex logic. A few utility functions could use additional documentation.
Overall Score	4.75 / 5	OptiCrop demonstrates excellent code quality with clean, modular, and well-

		documented Python and JavaScript code.
--	--	--